# Bilinear#

https://pytorch.org/docs/stable/generated/torch.nn.Bilinear.html#torch.nn.Bi

**Description:**

Applies a bilinear transformation to the incoming data: y=x1TAx2+by = x_1^T A x_2 + by=x1TAx2+b. in1_features (int) – size of each first input sample, must be > 0 in2_features (int) – size of each second input sample, must be > 0 out_features (int) – size of each output sample, must be > 0 bias (bool) – If set to False, the layer will not learn an additive bias. Default: True

## Function Signature

```
class torch.nn.Bilinear(in1_features, in2_features,
out_features, bias=True, device=None, dtype=None)[source]#
Parameters
Shape:
Variables
extra_repr()[source]#
```

**Returns:**

Tensor

## Code Examples

```
>>> m = nn.Bilinear(20, 30, 40)
>>> input1 = torch.randn(128, 20)
>>> input2 = torch.randn(128, 30)
>>> output = m(input1, input2)
>>> print(output.size())
torch.Size([128, 40])
```

## Detailed Documentation

### Documentation

Applies a bilinear transformation to the incoming data: y=x1TAx2+by = x_1^T A x_2 + by=x1TAx2+b.

in1_features (int) – size of each first input sample, must be > 0

in2_features (int) – size of each second input sample, must be > 0

out_features (int) – size of each output sample, must be > 0

bias (bool) – If set to False, the layer will not learn an additive bias. Default: True

Input1: (∗,Hin1)(*, H_\text{in1})(∗,Hin1) where Hin1=in1_featuresH_\text{in1}=\text{in1_features}Hin1=in1_features and ∗*∗ means any number of additional dimensions including none. All but the last dimension of the inputs should be the same.

Input2: (∗,Hin2)(*, H_\text{in2})(∗,Hin2) where Hin2=in2_featuresH_\text{in2}=\text{in2_features}Hin2=in2_features.

Output: (∗,Hout)(*, H_\text{out})(∗,Hout) where Hout=out_featuresH_\text{out}=\text{out_features}Hout=out_features and all but the last dimension are the same shape as the input.

weight (torch.Tensor) – the learnable weights of the module of shape (out_features,in1_features,in2_features)(\text{out_features}, \text{in1_features}, \text{in2_features})(out_features,in1_features,in2_features). The values are initialized from U(−k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(−k,k), where k=1in1_featuresk = \frac{1}{\text{in1_features}}k=in1_features1

bias – the learnable bias of the module of shape (out_features)(\text{out_features})(out_features). If bias is True, the values are initialized from U(−k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(−k,k), where k=1in1_featuresk = \frac{1}{\text{in1_features}}k=in1_features1

Return the extra representation of the module.

Runs the forward pass.

Resets parameters based on their initialization used in __init__.